Mobile Robot Teleoperation: Husky A200

M. J. Khojasteh

November 2026

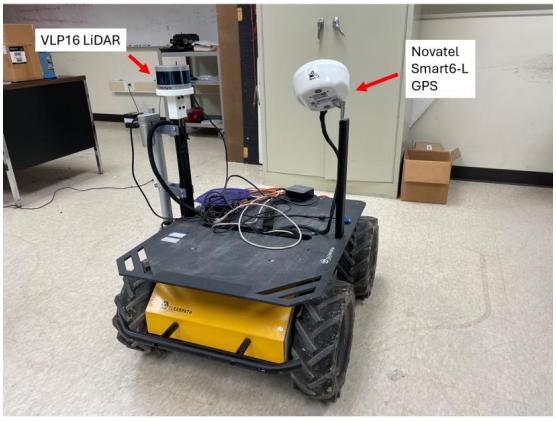
1 Objective

This lab uses the Clearpath Husky A200, a 50 kg four-wheel skid-steer robot built for outdoor work. It is much heavier and faster than the AmigoBot from lab 5, and that changes how carefully the software that drives it has to be written.

ROS 2 ships with a keyboard teleoperation node. It works, but it is not safe enough for a robot of this size: its speed keys have no upper limit, and it does nothing on its own to stop the robot when you let go of the keys. Your team will write its own teleop node as a new ROS 2 package, test it in simulation, put it in a Docker container, and hand it to the TA, who runs it on the real Husky while you drive.

Upon completion of this lab, students should be able to:

1. Explain how the Husky decides which of several velocity commands to obey, and why a teleop node still needs its own safety features.
2. Create a ROS 2 Python package with a teleop node that has a speed limit and a deadman (stop-on-release), and verify it in simulation.
3. Package a ROS 2 node in a Docker container that is configured from outside, without changing the code.
4. Compare the robot's behavior in simulation with the real robot, and explain the differences.



Technical Specifications:

Size: 990 × 670 × 390 mm;
Weight: 50 kg; **Payload**: up to 75 kg;
Max speed: 1.0 m/s;
Drive: 4 wheels, skid-steer;
Sensors (our robot): Velodyne VLP-16 LiDAR, UM6 IMU, Intel RealSense D435 camera, NovAtel GPS, wheel encoders;
Onboard computer: Ubuntu 22.04, ROS 2 Humble;
Safety: emergency stop on the status panel.

2 Pre-Lab

There is no pre-lab for this lab exercise. Bring a laptop on which the lab 4 container still works; the lab 6 container is built on the same base image, so it builds quickly.

3 Background

3.1 Who gets to drive the robot

The Husky's velocity controller listens to one command topic, but several programs may want to drive the robot (a joystick, a radio remote, your node). A node called `twist_mux` sits in front of the controller and decides. Each of its inputs has a *priority* and a *timeout*: it forwards the highest-priority input that has received a message within its timeout and ignores the rest. The emergency stop acts as a *lock* that silences all of them. Table 1 shows the configuration read from our robot. All topics live under the robot's namespace, `/a200_0252`.

Input	Topic (under `/a200_0252/`)	Priority	Timeout
Emergency stop (lock)	`platform/emergency_stop`	255	–
Radio remote	`rc_teleop/cmd_vel`	12	0.5 s
Logitech joystick (TA)	`joy_teleop/cmd_vel`	10	0.5 s
Interactive marker (RViz)	`twist_marker_server/cmd_vel`	8	0.5 s
Other programs (your node)	`cmd_vel`	1	0.5 s

Table 1: Velocity inputs of `twist_mux` on our Husky.

The velocity controller has a timeout of its own as well (`cmd_vel_timeout`, 0.5 s): if no command reaches it in that time, it stops the wheels.

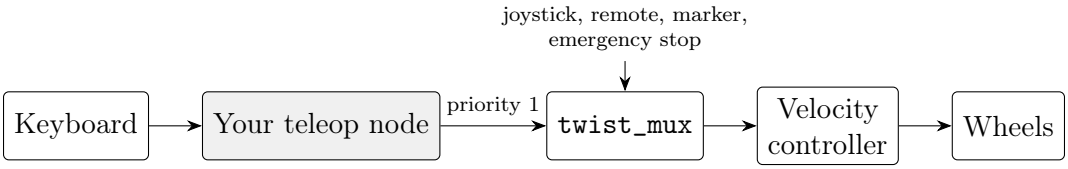


Figure 1: Path of a velocity command from the keyboard to the wheels.

Think about

The timeouts stop the robot if your node goes silent. What do they do if your node keeps publishing, but the person at the keyboard has stopped pressing keys?

3.2 Keyboards and the deadman

A terminal tells a program when a key is pressed. It never says when the key is released. If you hold a key down, the operating system starts repeating it, but only after a short delay (typically 0.25 to 0.6 s), and then about 30 times per second. So a teleop node has to decide for itself when the operator has let go.

Industrial machines solve this with a *deadman* control: motion continues only while the operator keeps asking for it, and stops on its own when they stop. In a teleop node, that means: if no motion key has arrived for a certain time, command zero. Picking that time is a trade-off you will have to make.

3.3 Containers for running code, not only developing it

In earlier labs you used a container as a development environment. Here you will also use one to hand your node to someone else. A **Dockerfile** is the recipe, an **image** is what `docker build` produces from it, and a **container** is a running copy of the image. Settings that change between the simulator and the robot, such as the speed limit or the topic name, should not be written into the code. They are passed in from outside when the container starts, as environment variables, so the same image runs everywhere.

4 Lab Setup

4.1 Safety rules

Our Husky runs Clearpath's ROS 2 software on its own computer. You will not log in to it. Your code runs in a container on the TA's robot laptop, which is wired to the robot.



Figure 2: Husky status panel. The red button on the left is the emergency stop: push to stop, twist in the direction of the arrows to release. The STOP and ERR lights stay on while it is pressed. COMM must be green before the robot can be driven.

These rules apply every time the robot is powered on:

1. The TA is in charge of the robot. When the TA says stop, everyone stops.
2. One team member stands beside the status panel with a hand near the emergency stop whenever a node is running. Anyone may call "stop" at any time.
3. The driver watches the robot, not the screen.
4. Nobody walks into the taped area while the robot can move.
5. The TA holds the Logitech joystick, which outranks your node. Students do not use it.

4.2 Workspace setup

1. Open the supplied `lab6/` folder in VS Code and run **Dev Containers: Reopen in Container**, picking your operating system as in lab 4. The `README.md` in that folder has the details.
2. Set up the display for RViz the same way as in lab 4.
3. New terminals start in `~/ros2_ws` with ROS 2 already sourced. It is the same folder as `lab6/ros2_ws` in the repository, so files you create there are also on your laptop. You will create your package in its `src` folder.

Start the simulated Husky in one terminal and leave it running:

Terminal

```
ros2 launch husky_lab sim.launch.py
```

It is a simple model, not a physics simulator, but it has the same command and odometry topics, the same `twist_mux` configuration and the same timeouts as our robot, and it shows the robot in RViz together with the taped line and stopping mark from the lab floor. In a second terminal, drive it with the stock ROS 2 teleop node:

Terminal

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard \
  --ros-args -r cmd_vel:=/a200_0252/cmd_vel
```

The container also includes a checker that runs your node, presses keys and measures what it publishes:

Terminal

```
lab6-check
```

Your team must pass the checker before your code goes on the robot.

5 Lab Tasks

The robot is shared by all teams, so plan to finish Tasks 1 to 3 in about an hour and a quarter. Teams go to the robot in the order they finish. A rough plan: Task 1 about 10 minutes, Task 2 about 50, Task 3 about 15.

Task 1: Explore the Interface

With the simulator running, find the following from the running system:

- Which topic does your node publish to, and what is its message type?
- Confirm the priorities and timeouts in Table 1.
- Drive with the stock teleop node while watching its output topic. When does it publish, and when does it not?
- Send a single velocity command with `ros2 topic pub --once`, then the same command repeatedly at 10 Hz. What does the robot do in each case? How long does it keep moving after the last message?

Useful tools

`ros2 topic list | info | echo | pub`, `ros2 param dump`, `rqt_graph`.

Task 2: Write Your Teleop Node

Create a Python package named `husky_teleop` in `~/ros2_ws/src`, with an executable named `teleop`. Use exactly these names: the checker and the TA's commands look for them. Your node uses the keys in Table 2 and must meet the requirements in Table 3.

Key	Command
`i` / `,`	forward / backward at the linear limit
`j` / `l`	turn left / right on the spot at the angular limit
`u` / `o`	forward while turning left / right (optional)
space, any other key	stop

Table 2: Keys for your teleop node.

ID	Requirement
R1	**Speed limit.** The linear and angular speed never exceed the limits in the environment variables `HUSKY_MAX_LINEAR` and `HUSKY_MAX_ANGULAR` (defaults 0.3 m/s and 0.6 rad/s).
R2	**Topic.** The node publishes `geometry_msgs/Twist` on the topic in `HUSKY_CMD_TOPIC` (default `/a200_0252/cmd_vel`).
R3	**Stop on release.** When no motion key has been pressed for a while, the node publishes zero. The robot must be commanded to stop within 0.6 s of the last key press, and must not stutter while a key is held down.
R4	**Stop keys.** The space bar, or any key that is not a motion key, stops the robot immediately. Ctrl-C publishes zero before the node exits.

Table 3: Requirements for your teleop node.

Reading single key presses from a terminal is fiddly and not the point of this lab, so it is done for you. The `lab6/starter` folder has two files:

- `keyboard.py`, a helper to copy into your package next to your node. Inside `with Keyboard() as kb:`, the call `kb.get_key()` returns the last key pressed since the previous call, or `None` if there was none. It never waits. The terminal is restored when the block ends.
- `teleop_main.py`, the `main()` function below and the imports it needs. Copy both into your node file. It makes sure your node publishes a zero command when it exits, provided you write the `stop()` method.

Given code: `main()`

```
def main():
    rclpy.init(signal_handler_options=SignalHandlerOptions.NO)
    with Keyboard() as kb:
        node = TeleopNode(kb)       # your class
        try:
            rclpy.spin(node)
        except KeyboardInterrupt:
            pass
        finally:
            node.stop()             # your method: publish a zero command
            node.destroy_node()
            rclpy.shutdown()
```

Notice that `rclpy.spin()` only runs your node's callbacks; nothing in `main()` reads the keyboard or publishes. Your node decides when to look for a key and when to publish. Once the package exists, build and run it with:

Terminal

```
colcon build --packages-select husky_teleop
source install/setup.bash
ros2 run husky_teleop teleop
```

When your node passes `lab6-check`, measure two stopping distances in the simulator. Drive along the taped line at 0.3 m/s and stop just as the front of the robot reaches the red mark: (a) with the space bar, and (b) by letting go of the key. The stopping distance is how far past the mark the front of the robot ends up. Read the robot's final position from its odometry, which gives the centre of the robot, 0.5 m behind its front bumper. Do each one twice. These are your predictions for the real robot.

Useful tools

`ros2 pkg create --help` (look at `--build-type`, `--dependencies`, `--node-name`); `entry_points` in `setup.py`; `Node.create_publisher`, `Node.create_timer`; `os.environ`; `time.monotonic()`; `ros2 topic echo` and `ros2 topic hz` to see what your node really publishes.

Think about

- If your stop-on-release time is very short, what happens when you hold a key down? If it is long, what happens when you let go? Which value did you choose, and why?
- Your node now publishes continuously. Go back to the question in Section 3.1. Why is R3 now your job and not the robot's?

Extra credit. Limit how fast the robot speeds up (for example, 0.5 m/s^2), but not how fast it stops. Explain why that is a sensible choice for a 50 kg skid-steer robot.

Task 3: Put Your Node in a Container

Write a `Dockerfile` in your package folder (`lab6/ros2_ws/src/husky_teleop`) that builds your package and starts your node. Run the Docker commands from a terminal on your laptop itself (Terminal on macOS, PowerShell on Windows), in that folder; the dev container does not have Docker inside it. On the robot laptop, the TA will build your image and start it with exactly these commands:

Terminal

```
docker build -t husky-teleop-team<N> .
docker run --rm -it --network host --ipc host \
  -e ROS_DOMAIN_ID=0 \
  -e HUSKY_CMD_TOPIC=/a200_0252/cmd_vel \
  -e HUSKY_MAX_LINEAR=0.1 -e HUSKY_MAX_ANGULAR=0.2 \
  husky-teleop-team<N>
```

To test it on your own laptop, keep the simulator running in the dev container and start your image without the robot settings. Your container and the dev container then share Docker's private network on your laptop, and the defaults in your code do the rest:

Terminal

```
docker run --rm -it husky-teleop-team<N>
```

Check that you can still drive the simulated robot. Try a different speed limit in the `docker run` command without rebuilding. Then stop the running container from another terminal with `docker stop` and time how long it takes.

When it works, show the TA, zip your `husky_teleop` folder and copy it onto the TA's USB stick.

Useful tools

`docker build`, `docker run`, `docker stop`, `docker ps`. Dockerfile instructions: `FROM` (start from `ros:humble-ros-base`), `COPY`, `RUN`, `CMD`.

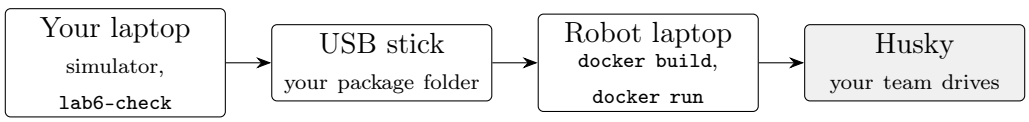


Figure 3: How your code gets to the robot.

Task 4: Test on the Husky

Each team has about 12 minutes with the robot. Decide beforehand who drives, who stands at the emergency stop, and who measures. Swap roles between steps.

1. **Low speed** (limits 0.1 m/s and 0.2 rad/s). Tap a key, hold a key, let go, press the space bar, and press Ctrl-C while moving. If the robot does not stop when it should, the test ends there: go back to the simulator, fix it, and ask the TA for another try if there is time.

2. **Stopping distance** (limits 0.3 m/s and 0.6 rad/s). Drive along the taped line and stop just as the front bumper reaches the mark: twice with the space bar, twice by letting go. Each time, measure from the mark to the front bumper with the tape measure.

3. **Drive the course.** Weave through the cones and park inside the taped box. The fastest clean run (no cones touched) goes on the board.

Task 5: Simulation Versus Reality

Compare the stopping distances you measured on the robot with your predictions from the simulator. This is done in your report.

6 Deliverables

1. Demonstrate your node passing `lab6-check` in the simulator.

2. Demonstrate your container running in the simulator, and hand it to the TA.

3. Drive the Husky with your node: low-speed test, stopping distances, and the course.

7 Questions

1. From Task 1: how long did the simulated robot keep moving after a single message? Explain that time using Table 1.

2. What stop-on-release time did you choose, and why? What would happen with 0.1 s, and with 2 s?

3. Compare your simulator predictions with the stopping distances measured on the robot, for the space bar and for letting go. Explain the differences. From the difference between letting go and the space bar, estimate your node's stop-on-release time, and compare it with the value in your code.

4. The given `main()` calls `rclpy.init()` with `SignalHandlerOptions.NO`. Remove that argument, rebuild, drive forward and press Ctrl-C. What is the last command your node publishes, and why? On the real robot, what would stop the Husky, and after how long?

5. How long did `docker stop` take with your container, and why? Suppose a node publishes continuously, has no stop-on-release, and takes 10 seconds to stop after `docker stop`. What would the robot do?

6. Why are the speed limits and topic name passed as environment variables instead of being written in the code? Give one other setting you would handle the same way.

7. The TA's `docker run` uses `--network host`, but yours does not. Why does the robot laptop need it? (Think about how ROS 2 nodes find each other.)

8. What could this lab improve on, and where do you wish the explanation had been better?

8 Report

1. Submit a technical memo. A skeleton/guideline is available on MyCourses.

2. Submit all materials electronically via MyCourses.

3. Your submission must be packaged into a single `.zip` archive containing the lab report and code.

4. Late submissions will receive a 10% penalty per day late.

   - The first 10% penalty will be applied at 12:01 a.m. the day after the demo/report is due.

5. Include the following items specific to this lab:

   - Your interface findings from Task 1.
   - Your node's source code, a short description of how it works, and how you tested each requirement.
   - Your Dockerfile.
   - Your stopping-distance measurements, in simulation and on the robot.
   - Answers to all of the questions.